Big Data Solution for Warsaw Public Transit delay prediction - Milestone 1 report

Big Data Analytics

Emil Łasocha Michał Szewczak Sebastian Trojan Tymoteusz Urban

October 18, 2026

1 Datasources

Data used in the project will be obtained from data published by the city of Warsaw through its open data APIs, available at (`https://dane.um.warszawa.pl`), but with multiple endpoints of both streaming and batch nature. Polled data is returned in json format for all endpoints available. All data size estimations where aquired using models available in Gemini free tier.

1.1 Streaming data

- **Vehicle position data**live GPS pings for every active vehicle: `Lat`, `Lon`, `Time`, `Lines`, textttBrigade, `VehicleNumber`. During rush hour there are about 450 trams and 1400 buses deployed, and such information is refreshed every 10s, though we would probably poll it in larger intervals, in order not to overload our limited infrastructure and not to exceed potential rate-limiting. A result of single poll has estimated size of 200KB.
- **Weather data** - air and surface temperature, precipitation,humidity, wind, and visibility for 18 stations located throughout the city. This data is refreshed every 10 minutes, though in order to use it as a streaming data source it is possible to poll it in shorter intervals for this puprose. A result of a single poll is estimated size of 4KB.
- **Air quality data** - returns a json with short description, monitoring station parameters, including type and location, and air quality parameters in form of nested records for 108 stations located throughout the city. This data is refreshed every 10 minutes, though in order to use it as a streaming data source it is possible to poll it in shorter intervals for this puprose. A result of a single poll is estimated size of 120KB.

1.2 Batch data

- **Timetable data** (static data, which should be acquired once per day) - stop locations, line-stop assignments, and per-brigade precise schedules for every stop-line-brigade combination in the network. This data daily (assuming daily batches) estimated size is at 140-200MB.

2 Use of Artificial Intelligence

AI tools will be used throughout the project to support software development, technical problem-solving, and report preparation. Their use will focus on improving productivity and assisting the team while ensuring that **all generated content and code are reviewed and verified by the team members**.

The planned uses of AI include:

- **Code development and editing:** Using AI tools integrated into the IDE to assist with writing, editing and debugging the code.
- **Technical support:** Using AI to clarify technical concepts, explain unfamiliar technologies, suggest implementation approaches, and assist in troubleshooting errors.
- **Report writing and editing:** Using AI to help structure the report, improve clarity and readability, correct grammatical and linguistic errors, and refine technical descriptions.
- **Verification of results:** Reviewing and testing AI-generated code, suggestions, and written content to ensure their correctness, relevance, and consistency with the actual project implementation.

AI will serve as a supporting tool rather than a replacement for the team's own work. All team members will remain responsible for understanding, validating, and maintaining the code and documentation they produce.

3 Division of Responsibilities

The project work is divided among four team members, each responsible for a specific component of the system and supported by another team member.

3.1 Data Ingestion and Validation

Main responsible person: Michał, supporting team member: Sebastian.

- Selecting and testing the weather API and public transport data source.
- Creating scripts in order to retrieve data from external APIs.
- Configuring Apache NiFi to automate the data ingestion process.
- Configuring Apache Kafka topics, such as `weather` and `buses`.
- Implementing data validation, error handling and duplicate detection.
- Defining and documenting data schemas.
- Documenting the data collection process.

Expected outcome: A working data ingestion pipeline that retrieves, validates, and delivers data to Apache Kafka.

3.2 Data Storage and Batch Processing

Main responsible person: Emil, supporting team member: Michał.

- Configuring Apache Hadoop, HDFS and Apache Hive.
- Designing a directory structure for raw, cleaned, and processed data.
- Creating Hive tables and developing SQL queries.
- Cleaning the collected data and handling missing or invalid values.
- Combining weather and public transport data into a unified dataset.
- Implementing batch processing jobs using Apache Spark.
- Calculating historical statistics, such as average bus delays by route and weather conditions.

Expected outcome: A functional data storage layer, a unified dataset, and a repeatable batch processing pipeline for historical analysis.

3.3 Stream Processing and Machine Learning

Main responsible person: Tymoteusz, supporting team member: Emil.

- Configuring Apache Spark Structured Streaming.
- Implementing the consumption of events from Apache Kafka.
- Calculating real-time metrics and statistics using streaming data.
- Preparing features from weather and public transport data for machine learning.
- Training a machine learning model to predict bus delays.
- Evaluating model performance with metrics.
- Integrating the trained model into the streaming pipeline to generate predictions.

Expected outcome: A working stream processing pipeline capable of calculating real-time metrics and generating bus delay predictions using a trained ML model.

3.4 Results Serving, Dashboard, and System Integration

Main responsible person: Sebastian, supporting team member: Tymoteusz .

- Configuring a database and creating tables for analytical results and predictions.
- Defining common data formats and interfaces with the other team members.
- Integrating the results of batch and stream processing.
- Developing an interactive dashboard using Streamlit.
- Visualising weather conditions, bus delays, historical statistics, and machine learning predictions.
- Integrating system components using Docker Compose.
- Testing the overall system and resolving integration issues.
- Preparing instructions for setting up and running the complete system.

Expected outcome: A functional dashboard presenting the results of the project, an integrated system, and documentation explaining how to run the application.